

Homework 03

Haoran Xu

2026-09-13

Question 1 (Build an explain-to-me skill)

Solution

I wrote `explain-to-me/SKILL.md` (reproduced verbatim in the appendix) and installed it by copying the folder into the agent’s personal skills directory, `~/claude/skills/explain-to-me/`, which is where Claude Code discovers user-level skills. The YAML front matter carries the `name` and a `description` that restricts the skill to explicit invocation, so it does not fire on ordinary requests and does not disturb my normal workflow.

The design goal was a skill that *explains and does not solve*. Rule 1 forbids answers, derivations carried to a result, and solution code; the remaining rules ask for a plain-language restatement of the goal, a checklist of deliverables, a list of traps, a pointer into one Week 3 lecture section, and a comprehension question back to me. A short output format keeps it usable.

I then started a fresh agent session and asked it to use `explain-to-me` on Question 2. It produced the five requested sections and no solution: it identified the fixed design and the 200 replications as the structure, listed the 24 summary numbers plus the two MSEs as the deliverables, and flagged exactly the traps I would have been most likely to get wrong — that “average squared value one” divides by n rather than $n - 1$, that $\mathbf{X}$ and μ are built once outside the loop while ϵ and ϵ^* are drawn inside it, that the penalty enters as $n\lambda$, and that the intercept used to predict $\mathbf{y}^*$ is the *training* mean $\bar{y}$. It pointed me to the “Put every covariate on a common scale” section and closed by asking which objects are regenerated in each replication. The skill behaved as intended, so I kept it installed.

Question 2 (Ridge regression with nearly collinear predictors)

Solution

Part a. (*design, collinearity diagnostics, and why the difference contrast is unstable.*)

```
set.seed(43231)
n <- 100; p <- 4
L <- rnorm(n); U1 <- rnorm(n); U2 <- rnorm(n); X3 <- rnorm(n); X4 <- rnorm(n)
X <- cbind(L + 0.04 * U1, L + 0.04 * U2, X3, X4)
X <- scale(X, center = TRUE, scale = FALSE)           # mean zero
X <- sweep(X, 2, sqrt(colMeans(X^2)), "/")           # average squared value one
d <- svd(X)$d
c(cor12 = cor(X[, 1], X[, 2]), d_max = d[1], d_min = d[p], kappa = d[1] / d[p])
#>      cor12      d_max      d_min      kappa
#> 0.997706 14.141480 0.475357 29.749148
```

The first two standardized columns have correlation 0.9977; the largest and smallest singular values are $d_{\max} = 14.14$ and $d_{\min} = 0.475$, a condition number of 29.8 (a ratio of 886 on the squared scale that governs variances). Writing $\mathbf{X}^T \mathbf{X} = \sum_j d_j^2 \mathbf{v}_j \mathbf{v}_j^T$, the OLS variance of a contrast $\mathbf{c}^T \hat{\beta}$ is $\sigma^2 \mathbf{c}^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{c} = \sigma^2 \sum_j (\mathbf{c}^T \mathbf{v}_j)^2 / d_j^2$, so a contrast is unstable exactly when it loads on a small singular value.

```
S <- solve(crossprod(X)); ev <- eigen(crossprod(X))
c(var_sum = sum(S[1:2, 1:2]), var_diff = S[1, 1] + S[2, 2] - 2 * S[1, 2])
```

```
#> var_sum var_diff
#> 0.010022 8.850626
rbind(v_max = round(ev$eigenvectors[, 1], 3), v_min = round(ev$eigenvectors[, p], 3))
#>      [,1] [,2] [,3] [,4]
#> v_max -0.706 -0.706 -0.046 0.004
#> v_min  0.707 -0.707  0.001 0.006
```

Because X_1 and X_2 are both L plus a small perturbation, the top right singular vector is essentially $(1, 1, 0, 0)/\sqrt{2}$ and the bottom one is essentially $(1, -1, 0, 0)/\sqrt{2}$. Hence the sum contrast sits almost entirely in the well-determined direction, with variance $\approx 2/d_{\max}^2 = 0.010$, while the difference contrast sits almost entirely in the flat direction, with variance $\approx 2/d_{\min}^2 = 8.85$ — about 880 times larger. The data pin down $\beta_1 + \beta_2$ very precisely and say almost nothing about $\beta_1 - \beta_2$.

Part b. (200 replications at three penalties on the fixed design.)

```
beta <- c(1.5, 1.5, 1, 0); mu <- as.vector(X %*% beta)
lams <- c(0, 0.02, 0.2); B <- 200; XtX <- crossprod(X)
out <- array(NA, c(B, 6, 3),
             dimnames = list(NULL, c("b1", "b2", "sum", "diff", "train", "test"), NULL))
for (b in 1:B) {
  y <- mu + rnorm(n); ystar <- mu + rnorm(n) # both drawn inside the loop
  b0 <- mean(y); Xty <- crossprod(X, y - b0) # unpenalized intercept
  for (l in 1:3) {
    bh <- solve(XtX + n * lams[l] * diag(p), Xty) # reuse the same (y, y*) pair
    f <- b0 + as.vector(X %*% bh)
    out[b, , l] <- c(bh[1], bh[2], bh[1] + bh[2], bh[1] - bh[2],
                    mean((y - f)^2), mean((ystar - f)^2))
  }
}
tab <- t(sapply(1:3, function(l) {
  m <- colMeans(out[, , l]); s <- apply(out[, , l], 2, sd)
  c(lambda = lams[l], m["b1"], sd1 = s["b1"], m["b2"], sd2 = s["b2"],
    m["sum"], sdS = s["sum"], m["diff"], sdD = s["diff"], m["train"], m["test"])
}))
round(tab, 4)
#>      lambda      b1 sd1.b1      b2 sd2.b2      sum sdS.sum      diff sdD.diff
#> [1,]  0.00 1.4956 1.5439 1.5098 1.5486 3.0054 0.0951 -0.0142 3.0911
#> [2,]  0.02 1.4870 0.1615 1.4892 0.1659 2.9762 0.0942 -0.0022 0.3136
#> [3,]  0.20 1.3679 0.0457 1.3687 0.0473 2.7366 0.0865 -0.0008 0.0344
#>      train test
#> [1,] 0.9648 1.0625
#> [2,] 0.9747 1.0493
#> [3,] 1.0768 1.1335
```

Table 1: Empirical mean (standard deviation) over 200 replications, plus average MSEs.

λ	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_1 + \hat{\beta}_2$	$\hat{\beta}_1 - \hat{\beta}_2$	train MSE	test MSE
0 (OLS)	1.496 (1.544)	1.510 (1.549)	3.005 (0.095)	-0.014 (3.091)	0.965	1.062
0.02	1.487 (0.162)	1.489 (0.166)	2.976 (0.094)	-0.002 (0.314)	0.975	1.049
0.2	1.368 (0.046)	1.369 (0.047)	2.737 (0.087)	-0.001 (0.034)	1.077	1.134

Part c. (bias–variance reading.) Under OLS the individual slopes are unbiased — their means, 1.496 and 1.510, sit on the truth 1.5 — but their standard deviation is 1.54, so a single data set can easily report $\hat{\beta}_1 = 3$ and $\hat{\beta}_2 = 0$. All of that instability lives in the difference contrast (SD 3.09), matching the predicted $\sqrt{8.85} = 2.98$; the sum is already stable (SD 0.095 versus the predicted $\sqrt{0.010} = 0.100$). Ridge at $\lambda = 0.02$ multiplies the j th eigendirection of the coefficient by $d_j^2/(d_j^2 + n\lambda)$: for $d_{\min}^2 = 0.226$ that factor is 0.184, so the difference SD collapses by a factor

of ten to 0.314, while for $d_{\max}^2 = 200$ the factor is 0.990 and the sum is essentially untouched. The bias paid is tiny (1.487 versus 1.5) and the test MSE improves from 1.062 to 1.049. At $\lambda = 0.2$ the shrinkage reaches the well-determined direction too: the sum falls to 2.737, a clear bias, and although the variance is now negligible the test MSE rises to 1.134.

The reason ridge can stabilize coefficients so much without moving the fitted values much is that the flat direction contributes almost nothing to $\hat{\mu}$. A coefficient perturbation $\delta \mathbf{v}_{\min}$ changes the fit by $\|\mathbf{X} \delta \mathbf{v}_{\min}\| = |\delta| d_{\min} = 0.475|\delta|$, so wild swings in $\beta_1 - \beta_2$ barely register in $\mathbf{X}\hat{\beta}$ — which is also why the data cannot identify that contrast in the first place. Ridge removes variance where the design is blind and the predictions are insensitive, which is nearly free; the training MSE only rises from 0.965 to 0.975. Prediction gains are therefore modest here, while the gain in interpretability of the individual slopes is large.

Question 3 (What does ridge regression shrink?)

Solution

Part a. (normal equation and uniqueness.) With $L_\lambda(\beta) = \frac{1}{2n} \|\tilde{\mathbf{y}} - \mathbf{X}\beta\|_2^2 + \frac{\lambda}{2} \|\beta\|_2^2$,

$$\nabla L_\lambda(\beta) = -\frac{1}{n} \mathbf{X}^\top (\tilde{\mathbf{y}} - \mathbf{X}\beta) + \lambda \beta = \frac{1}{n} \{(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \beta - \mathbf{X}^\top \tilde{\mathbf{y}}\}.$$

Setting the gradient to zero gives $(\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \hat{\beta}_\lambda = \mathbf{X}^\top \tilde{\mathbf{y}}$. The Hessian is $\nabla^2 L_\lambda = \frac{1}{n} (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)$, a constant matrix. For any $\mathbf{z} \neq \mathbf{0}$, $\mathbf{z}^\top (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p) \mathbf{z} = \|\mathbf{X}\mathbf{z}\|_2^2 + n\lambda \|\mathbf{z}\|_2^2 \geq n\lambda \|\mathbf{z}\|_2^2 > 0$ whenever $\lambda > 0$: every eigenvalue of $\mathbf{X}^\top \mathbf{X}$ is ≥ 0 , and adding $n\lambda$ lifts all of them, including the zero ones, strictly above zero. So the matrix is positive definite, hence invertible, and L_λ is strictly convex with the unique minimizer $\hat{\beta}_\lambda = (\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p)^{-1} \mathbf{X}^\top \tilde{\mathbf{y}}$. This holds no matter how $\mathbf{X}$ is shaped: rank deficiency or $p > n$ only means $\mathbf{X}^\top \mathbf{X}$ has zero eigenvalues, which the ridge term repairs. OLS ($\lambda = 0$) has no such guarantee.

Part b. (shrinkage factors from the SVD.) Write $\mathbf{X} = \mathbf{U}_r \mathbf{D}_r \mathbf{V}_r^\top$, so $\mathbf{X}^\top \mathbf{X} = \mathbf{V}_r \mathbf{D}_r^2 \mathbf{V}_r^\top$ and $\mathbf{X}^\top \tilde{\mathbf{y}} = \mathbf{V}_r \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}$, which lies in the column space of $\mathbf{V}_r$. Complete $\mathbf{V}_r$ to an orthonormal basis with $\mathbf{V}_\perp$ spanning the null space of $\mathbf{X}$. In that basis $\mathbf{X}^\top \mathbf{X} + n\lambda \mathbf{I}_p$ is block diagonal, equal to $\mathbf{D}_r^2 + n\lambda \mathbf{I}_r$ on $\mathbf{V}_r$ and $n\lambda \mathbf{I}$ on $\mathbf{V}_\perp$. Since the right-hand side has no $\mathbf{V}_\perp$ component,

$$\hat{\beta}_\lambda = \mathbf{V}_r (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{D}_r \mathbf{U}_r^\top \tilde{\mathbf{y}}, \quad \mathbf{X} \hat{\beta}_\lambda = \mathbf{U}_r \mathbf{D}_r^2 (\mathbf{D}_r^2 + n\lambda \mathbf{I}_r)^{-1} \mathbf{U}_r^\top \tilde{\mathbf{y}},$$

using $\mathbf{V}_r^\top \mathbf{V}_r = \mathbf{I}_r$. Reading this coordinate by coordinate,

$$\mathbf{X} \hat{\beta}_\lambda = \sum_{j=1}^r \frac{d_j^2}{d_j^2 + n\lambda} \mathbf{u}_j (\mathbf{u}_j^\top \tilde{\mathbf{y}}),$$

so the component of the fitted response along $\mathbf{u}_j$ is the OLS component $\mathbf{u}_j^\top \tilde{\mathbf{y}}$ multiplied by $\rho_j(\lambda) = d_j^2 / (d_j^2 + n\lambda)$, as claimed. At $\lambda = 0$ every $\rho_j = 1$ and the projection is exact.

```
dj <- c(10, 2, 0.2); n <- 100; lambda <- 0.01
rho <- dj^2 / (dj^2 + n * lambda)
c(rho = rho, df_eff = 1 + sum(rho))
#>      rho1      rho2      rho3      df_eff
#> 0.9900990 0.8000000 0.0384615 2.8285605
```

With $n\lambda = 1$: $\rho_1 = 100/101 = 0.9901$, $\rho_2 = 4/5 = 0.8$, and $\rho_3 = 0.04/1.04 = 0.03846$, giving $\text{df}_{\text{eff}}(\lambda) = 1 + 1.8286 = 2.8286$ out of the $p + 1 = 4$ coefficients that OLS would spend.

Part c. (which direction is shrunk, and when that hurts.) The strongest shrinkage falls on $\mathbf{u}_3$, the direction with the smallest singular value $d_3 = 0.2$: it is multiplied by 0.038, so 96% of its OLS contribution is removed, whereas $\mathbf{u}_1$ keeps 99%. The rule is $\rho_j \approx d_j^2 / (n\lambda)$ once $d_j^2 \ll n\lambda$, so ridge is aggressive precisely where the design carries little variance. That reduces prediction variance because the OLS coefficient along $\mathbf{v}_j$ has variance σ^2 / d_j^2 , which is $\sigma^2 / 0.04 = 25\sigma^2$ here versus $\sigma^2 / 100$ along $\mathbf{v}_1$. The direction is estimated almost entirely from noise, and multiplying a mostly-noise term by 0.038 removes essentially all of the variance it contributes while costing little in fit.

The same shrinkage produces substantial bias when the true mean has a real, sizeable component in that low-variance direction, that is when $|\mathbf{u}_j^\top \mu|$ is large even though d_j is small. Then the signal is genuine but weak in the design, ridge discards 96% of it, and the squared bias $(1 - \rho_j)^2 (\mathbf{u}_j^\top \mu)^2$ can dominate the variance saved. Ridge implicitly assumes that directions the design explores poorly also carry little signal; when that assumption fails, so does the bias-variance trade.

Question 4 (Ridge regression and optimization)

Solution

Part a. (conditioning.) The eigenvalues of $\mathbf{A}_0$ are 9 and 0.01, so $\kappa(\mathbf{A}_0) = 9/0.01 = 900$. Adding $\lambda = 0.09$ shifts every eigenvalue by the same amount, giving 9.09 and 0.10, so $\kappa(\mathbf{A}_\lambda) = 9.09/0.10 = 90.9$, a tenfold improvement. Geometrically, the level sets of the least-squares objective are ellipses with axis-length ratio $\sqrt{900} = 30$ — a long, narrow valley that is nearly flat along the low-curvature direction, which is why the minimizer is poorly determined there. The ridge penalty adds the isotropic bowl $\frac{\lambda}{2} \|\beta\|_2^2$, which raises the floor of the valley without touching the steep direction in relative terms; the ellipses become much rounder (axis ratio $\sqrt{90.9} = 9.5$), and the objective acquires a strong-convexity constant of at least λ . The absolute change is entirely in the small eigenvalue, which grows by a factor of ten while the large one grows by 1%.

Part b. (gradient descent contraction.) From Question 3a, $\nabla L_\lambda(\beta) = \mathbf{A}_\lambda \beta - \frac{1}{n} \mathbf{X}^\top \tilde{\mathbf{y}}$, and the minimizer satisfies $\mathbf{A}_\lambda \hat{\beta}_\lambda = \frac{1}{n} \mathbf{X}^\top \tilde{\mathbf{y}}$. Subtracting, $\nabla L_\lambda(\beta^{(k)}) = \mathbf{A}_\lambda(\beta^{(k)} - \hat{\beta}_\lambda) = \mathbf{A}_\lambda \mathbf{e}^{(k)}$, so

$$\mathbf{e}^{(k+1)} = \beta^{(k)} - \eta \mathbf{A}_\lambda \mathbf{e}^{(k)} - \hat{\beta}_\lambda = (\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{e}^{(k)}.$$

Let $\mathbf{q}$ be a unit eigenvector of $\mathbf{A}_0$ with eigenvalue a . Then $\mathbf{A}_\lambda \mathbf{q} = (a + \lambda) \mathbf{q}$, so the eigendecompositions coincide and $(\mathbf{I}_p - \eta \mathbf{A}_\lambda) \mathbf{q} = \{1 - \eta(a + \lambda)\} \mathbf{q}$. Writing $\mathbf{e}^{(k)} = \sum_i e_i^{(k)} \mathbf{q}_i$, the coordinates decouple and $e_i^{(k+1)} = \{1 - \eta(a_i + \lambda)\} e_i^{(k)}$: each eigendirection is multiplied by $1 - \eta(a + \lambda)$ every step, independently of the others.

Every direction contracts iff $|1 - \eta(a + \lambda)| < 1$ for both eigenvalues, i.e. $0 < \eta < 2/(a + \lambda)$ for each, and the binding constraint is the largest eigenvalue $M = 9.09$. The admissible interval is therefore

$$0 < \eta < \frac{2}{M} = \frac{2}{9.09} = 0.2200.$$

Choice $\eta = 1/M = 0.11001$. The contraction factors are $1 - 9.09/9.09 = 0$ in the steep direction and $1 - 0.10/9.09 = 0.98900$ in the flat one. The steep direction converges immediately — a single step annihilates its error, because the step size is exactly matched to that curvature — while the flat direction contracts by only 1.1% per step and needs roughly $\log(10^{-6})/\log(0.989) \approx 1250$ iterations for six digits. Convergence is governed by the worst factor $1 - 1/\kappa(\mathbf{A}_\lambda) = 1 - 1/90.9$, so it is the low-curvature direction that dictates the run time.

Choice $\eta = 2.1/M = 0.23102$. Now the factor in the M direction is $1 - 2.1 = -1.1$, with $|-1.1| = 1.1 > 1$: that component alternates in sign and grows by 10% each step, so $|e_M^{(k)}| = 1.1^k |e_M^{(0)}| \rightarrow \infty$ since the initial error has a nonzero component there. The flat direction still contracts, with factor $1 - 0.23102(0.10) = 0.97690$, but one diverging coordinate is enough: the method diverges, overshooting the minimum along the steep axis and oscillating with growing amplitude. The step size exceeds $2/M$, which is exactly the stability limit derived above.

Part c. (why fast optimization is not a tuning rule.) The two criteria measure different things. The condition number $\kappa(\mathbf{A}_\lambda) = (a_{\max} + \lambda)/(a_{\min} + \lambda)$ decreases monotonically to 1 as $\lambda \rightarrow \infty$, so “choose λ for the fastest convergence” has the degenerate solution $\lambda = \infty$, whose minimizer is $\hat{\beta} = \mathbf{0}$ — a model that predicts $\bar{y}$ for everyone and converges in one step. Optimization speed asks how quickly we reach the minimizer of a *given* objective; statistical tuning asks which objective’s minimizer predicts best out of sample. λ controls bias against variance, and that trade-off has an interior optimum that only data-driven criteria such as cross-validation or GCV can locate, as in Question 5, where $\lambda_{\min} \approx 0.02$ while the conditioning would keep improving far beyond it. Conditioning is also the solver’s problem, not the model’s: with p small we solve the linear system directly and κ is irrelevant, and otherwise preconditioning, conjugate gradients, or momentum fix the geometry without changing the estimator. Letting a numerical convenience choose a statistical parameter conflates the two.

Question 5 (Selecting the ridge penalty for real-estate prediction)

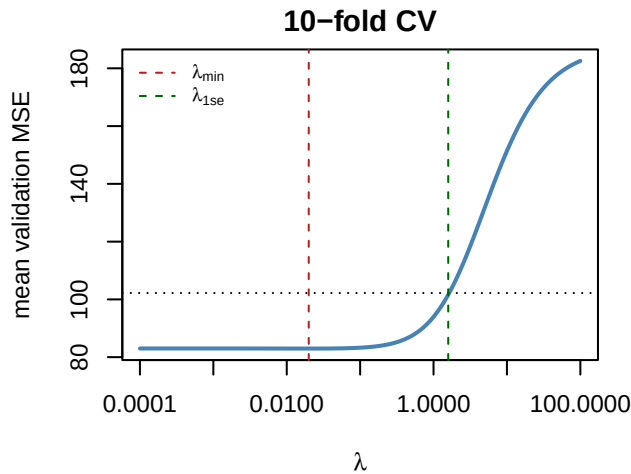
Solution

```
dat <- merge(read.csv("data/realestate.csv"),
             read.csv("data/realestate-split-folds.csv"), by = "row_id")
pv <- c("date", "age", "distance", "stores", "latitude", "longitude")
tr <- dat[dat$split == "train", ]; te <- dat[dat$split == "test", ]
lam <- c(0, 10^(-4 + 0.05 * (0:120))) # OLS plus 121 log-spaced penalties
scales <- function(rows) { # means and scales from fitting data only
  M <- as.matrix(rows[, pv]); m <- colMeans(M)
  list(m = m, s = sqrt(colMeans(sweep(M, 2, m, "-")^2))) # variance divided by n_fit
}
zs <- function(rows, p) sweep(sweep(as.matrix(rows[, pv]), 2, p$m, "-"), 2, p$s, "/")
rfit <- function(Z, y, l) { # unpenalized intercept = mean(y)
  b0 <- mean(y)
  list(b0 = b0, b = as.vector(solve(crossprod(Z) + nrow(Z) * 1 * diag(ncol(Z)),
                                     crossprod(Z, y - b0))))
}
pred <- function(f, Z) f$b0 + as.vector(Z %*% f$b)
c(n_train = nrow(tr), n_test = nrow(te), n_lambda = length(lam))
#>   n_train  n_test n_lambda
#>    332     82    122
```

Part a. (ten-fold cross-validation.)

```
cvE <- matrix(NA, 10, length(lam))
for (k in 1:10) {
  itr <- tr[tr$cv_fold != k, ]; iva <- tr[tr$cv_fold == k, ]
  p <- scales(itr); Zi <- zs(itr, p); Zv <- zs(iva, p) # scales from the nine folds only
  for (j in seq_along(lam))
    cvE[k, j] <- mean((iva$price - pred(rfit(Zi, itr$price, lam[j])), Zv))^2)
}
mcv <- colMeans(cvE); se <- apply(cvE, 2, sd) / sqrt(10)
jmin <- which.min(mcv); lam_min <- lam[jmin]
lam_1se <- max(lam[mcv <= mcv[jmin] + se[jmin]])
c(lam_min = lam_min, cv_min = mcv[jmin], se_min = se[jmin],
  threshold = mcv[jmin] + se[jmin], lam_1se = lam_1se, cv_1se = mcv[lam == lam_1se])
#>   lam_min  cv_min  se_min threshold  lam_1se  cv_1se
#> 0.0199526 82.9776494 19.2299137 102.2075631 1.5848932 101.5955317

par(mfrow = c(1, 2), mar = c(4, 4, 2, 1), cex = 0.75)
plot(lam[-1], mcv[-1], log = "x", type = "l", lwd = 2, col = "steelblue",
     xlab = expression(lambda), ylab = "mean validation MSE", main = "10-fold CV")
abline(h = mcv[jmin] + se[jmin], lty = 3)
abline(v = c(lam_min, lam_1se), lty = 2, col = c("firebrick", "darkgreen"))
legend("topleft", bty = "n", cex = 0.8, lty = 2, col = c("firebrick", "darkgreen"),
     legend = c(expression(lambda[min]), expression(lambda[1*se])))
```

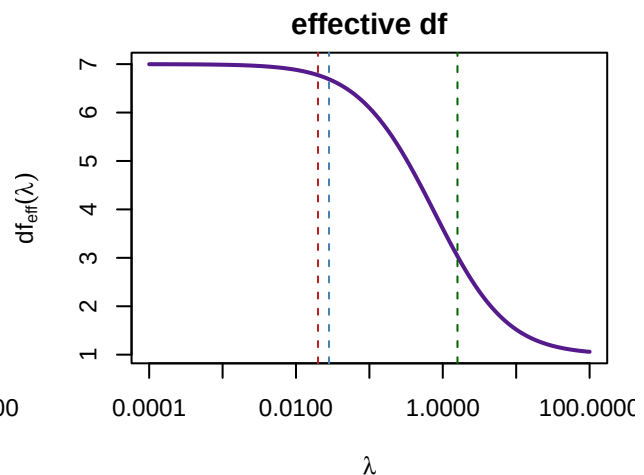
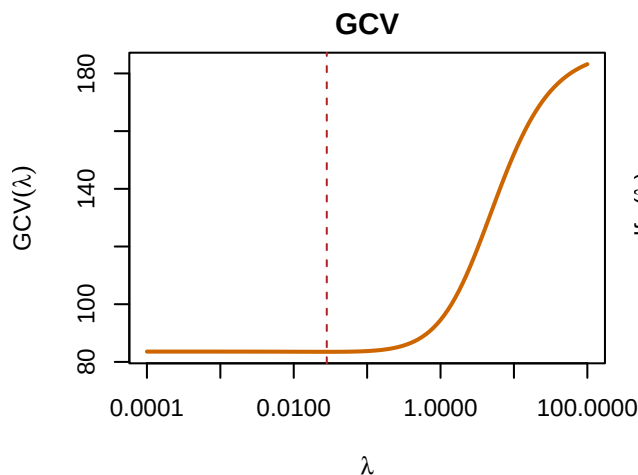


The validation curve is essentially flat over the whole lower range and turns up only past $\lambda \approx 0.3$. The minimum is at $\lambda_{\min} = 0.019953$ with mean validation MSE 82.978 and $SE(\lambda_{\min}) = 19.230$; OLS itself scores 83.012, so the penalty buys almost nothing here. The standard error is enormous relative to the spread of the curve — the ten fold errors disagree far more than the candidate models do — so the one-standard-error threshold 102.208 is satisfied all the way out to $\lambda_{1se} = 1.5849$, two orders of magnitude above $\lambda_{\min}$.

Part b. (GCV on all 332 training observations.)

```
p_all <- scales(tr); Za <- zs(tr, p_all); ya <- tr$price
n <- nrow(Za); dsv <- svd(Za)$d
dfv <- sapply(lam, function(l) 1 + sum(dsv^2 / (dsv^2 + n * l)))
trm <- sapply(lam, function(l) mean((ya - pred(rfit(Za, ya, l), Za))^2))
gcv <- trm / (1 - dfv / n)^2
lam_gcv <- lam[which.min(gcv)]
c(lam_gcv = lam_gcv, gcv = min(gcv), df = dfv[which.min(gcv)])
#>   lam_gcv      gcv      df
#> 0.0281838 83.5019140 6.6886567
```

```
par(mfrow = c(1, 2), mar = c(4, 4, 2, 1), cex = 0.75)
plot(lam[-1], gcv[-1], log = "x", type = "l", lwd = 2, col = "darkorange3",
     xlab = expression(lambda), ylab = expression(GCV(lambda)), main = "GCV")
abline(v = lam_gcv, lty = 2, col = "firebrick")
plot(lam[-1], dfv[-1], log = "x", type = "l", lwd = 2, col = "purple4",
     xlab = expression(lambda), ylab = expression(df_eff(lambda)), main = "effective df")
abline(v = c(lam_min, lam_1se, lam_gcv), lty = 2,
       col = c("firebrick", "darkgreen", "steelblue"))
```



GCV is minimized at $\lambda_{\text{GCV}} = 0.028184$ with $\text{GCV} = 83.502$ and $\text{df}_{\text{eff}} = 6.689$. This is remarkably close to λ_{min} , which is reassuring: GCV is an analytic approximation to leave-one-out CV computed from a single fit, and here it agrees with ten-fold CV to within one grid step, even though the two criteria use different data splits and different formulas. The design is well conditioned (singular values 29.6 down to 7.2, so $\kappa = 4.1$), so tiny penalties barely move df_{eff} away from $p + 1 = 7$ and the curve is very shallow near its minimum.

Part c. (*final refits on the 332 training observations.*) The three penalties were fixed above before any test response was touched.

```
Zt <- zs(te, p_all)
final <- t(sapply(c(OLS = 0, lam_min = lam_min, lam_1se = lam_1se, lam_GCV = lam_gcv),
  function(l) {
    f <- rfit(Za, ya, l)
    c(lambda = l, df_eff = 1 + sum(dsv^2 / (dsv^2 + n * l)),
      train = mean((ya - pred(f, Za))^2), test = mean((te$price - pred(f, Zt))^2))
  }))
round(final, 4)
#>      lambda df_eff   train   test
#> OLS      0.0000 7.0000  80.0939 66.3225
#> lam_min  0.0200 6.7732  80.1354 65.9996
#> lam_1se  1.5849 3.0329 100.3736 81.1998
#> lam_GCV  0.0282 6.6887  80.1713 65.9028
```

Table 2: Final models fitted on all 332 training observations and evaluated on the 82 held-out transactions.

model	λ	df_{eff}	training MSE	test MSE
OLS	0	7.000	80.094	66.323
λ_{min}	0.019953	6.773	80.135	66.000
λ_{1se}	1.584893	3.033	100.374	81.200
λ_{GCV}	0.028184	6.689	80.171	65.903

Training MSE increases monotonically in λ , as it must, since OLS minimizes it by construction and every penalty pulls the fit away from that optimum. Effective degrees of freedom fall the other way, from 7 for OLS to 3.03 at λ_{1se} : that model spends barely more than two slopes’ worth of flexibility on six predictors, and its coefficients are shrunk roughly threefold ($-5.92 \rightarrow -2.26$ for `distance`, for example). λ_{min} and λ_{GCV} are nearly indistinguishable from OLS on every column — with $n = 332$, $p = 6$, and a well-conditioned design, there is little estimation variance for ridge to remove, so the best available trade is almost no shrinkage at all. The mild penalties do improve test MSE slightly ($66.32 \rightarrow 65.90$), which is the expected small variance reduction, while λ_{1se} is clearly over-penalized: its test MSE of 81.20 is 23% worse, the bias it introduces swamping any variance it saves. The one-standard-error heuristic misfires here because $\text{SE}(\lambda_{\text{min}}) = 19.2$ is large compared with the entire range of the validation curve, so the rule is free to walk far up the λ axis without formally violating its own threshold; it is a rule calibrated for settings where the CV curve has a sharp, well-resolved minimum.

Finally, the model with the smallest observed test MSE must not now be selected on that basis. The test responses were held out to give one honest, unbiased estimate of generalization error for a model chosen without them. Choosing among the four rows by their test column would use the test set as a fourth selection criterion, and the reported 65.90 would then be a minimum over four correlated estimates, biased downward and no longer a valid statement about future data. With only 82 test observations the differences involved are well inside the noise: the gap between 66.32 and 65.90 is a fraction of a percent, far smaller than the standard error of either estimate, so it carries no evidence that one model is genuinely better. The defensible report is that λ_{GCV} (or equivalently λ_{min}) was selected by a criterion computed from training data alone, and that its out-of-sample MSE is approximately 65.9. Note too that GCV is not itself an independent assessment: it estimates prediction error by inflating the training residuals with df_{eff} , so the test set remains the only held-out evidence. And ridge only controls coefficient variance within this linear model; it does not repair nonlinearity, omitted variables, or other misspecification.

explain-to-me skill

The complete contents of explain-to-me/SKILL.md as installed:

```
---
name: explain-to-me
description: Explain what a STAT 432 homework question is asking without solving it. Use only
  ↳ when the student explicitly names "explain-to-me" and points to a question; never trigger
  ↳ it automatically.
---

# explain-to-me

My intent: I want to understand a homework question on my own terms before I
write any code or algebra. Explain the question to me; do not do it for me.

## Rules

1. Never produce the answer. No derivations carried to a result, no fitted
  numbers, no runnable solution code, no pseudo-code that is the solution in
  disguise. If I ask you to "just show me," restate this rule once and continue
  explaining.
2. Restate the goal in two or three plain sentences, naming what is given,
  what is random, what is held fixed, and what the question wants reported.
3. List the deliverables as a checklist - one line per quantity, plot, or
  claim the question asks for, including anything buried in a subclause.
4. Flag the traps: the scaling convention, what stays fixed across
  repetitions, which quantities are unpenalized, and any step whose order
  matters (e.g. fixing a tuning parameter before looking at test data).
5. Define the notation the question uses but does not spell out.
6. Point me to one relevant Week 3 section, by name and link, from
  [Ridge Regression: Stability Through Shrinkage](https://teazrq.github.io/stat432rpy/topic_
  ↳ s/ridge-regression/ridge-regression.html)
  or [From a Penalized Objective to a Fitted Ridge Model](https://teazrq.github.io/stat432r
  ↳ py/topics/ridge-regression/optimization-and-cross-validation.html),
  and say in one sentence why that section is the right one.
7. End with one question back to me that checks whether I have understood
  the part most likely to be misread.

## Output format

Keep it under roughly 300 words, in this order: Goal, What you must
produce, Watch out for, Where to read, Check yourself.
```